Software Engineering - GenAI Frontend

Caoimhe Coveney McKeown
25249131

GenAI was used as a development tool to assist with frontend implementation and development. It was primarily used to generate initial code structures and provide guidance on specific features such as UI components and responsive map design. All generated code was reviewed, adapted and extended with the final implementation being my own code developed using these outputs as a foundation.

Contents of chats:

- Leaflet map implementation with dynamic station data
- Marker colour logic based on bike availability
- Autocomplete search functionality for station names
- UI layout and responsive map overlay design

Can you explain how to design an interactive bike station map using Leaflet, including marker colouring, popups, and filtering based on availability?

You can design the interactive bike station map as a Leaflet-based page that loads live station data, displays each station as a marker, colours the marker by availability, and lets the user filter stations by name or bike availability.

Interactive Bike Station Map Using Leaflet

To create the map, first include the Leaflet CSS and JavaScript files in the HTML page. The map itself can be placed inside a `<div>` with an id such as `map`.

HTML

```html
<div id="map"></div>
```

The map is then initialised in JavaScript by setting a starting location and zoom level. For Dublin, the map can be centred around the city centre.

JavaScript

```javascript
const map = L.map("map").setView([53.3498, -6.2603], 13);

L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
  attribution: "&copy; OpenStreetMap contributors"
}).addTo(map);
```

Each bike station can be represented as a marker. The marker colour can be based on the number of available bikes:

JavaScript

```javascript
function getMarkerColour(availableBikes) {
  if (availableBikes >= 15) {
    return "green";
  } else if (availableBikes >= 5) {
    return "orange";
  } else {
    return "red";
  }
}
```

For a cleaner visual design, custom circle markers work well because they are easy to colour dynamically:

JavaScript

```javascript
function createStationMarker(station) {
  const colour = getMarkerColour(station.available_bikes);

  const marker = L.circleMarker([station.lat, station.lng], {
    radius: 8,
    fillColor: colour,
    color: "#ffffff",
    weight: 2,
    opacity: 1,
    fillOpacity: 0.9
  });

  marker.bindPopup(`
    <strong>${station.name}</strong><br>
    Available bikes: ${station.available_bikes}<br>
    Available stands: ${station.available_stands}<br>
    Status: ${station.status}
  `);

  return marker;
}
```

The popup gives the user more detail when they click a station. This is useful because the map itself can stay visually simple, while the popup provides station-specific information.

A basic station-rendering function could look like this:

JavaScript

```javascript
let stationMarkers = [];

function displayStations(stations) {
  stationMarkers.forEach(marker => map.removeLayer(marker));
  stationMarkers = [];

  stations.forEach(station => {
    const marker = createStationMarker(station);
    marker.addTo(map);
    stationMarkers.push(marker);
  });
}
```

Filtering can then be added using a text search box and an availability dropdown.

HTML

```html
<input id="stationSearch" type="text" placeholder="Search stations..." />

<select id="availabilityFilter">
  <option value="all">All stations</option>
  <option value="high">High availability</option>
  <option value="medium">Medium availability</option>
  <option value="low">Low availability</option>
</select>
```

The filtering logic checks whether the station name matches the search term and whether the availability level matches the selected filter.

JavaScript

```javascript
function getAvailabilityLevel(availableBikes) {
  if (availableBikes >= 15) {
    return "high";
  } else if (availableBikes >= 5) {
    return "medium";
  } else {
    return "low";
  }
}

function filterStations(stations) {
  const searchTerm = document
    .getElementById("stationSearch")
    .value
    .toLowerCase();

  const availability = document.getElementById("availabilityFilter").value;

  const filteredStations = stations.filter(station => {
    const matchesSearch = station.name.toLowerCase().includes(searchTerm);
    const matchesAvailability =
      availability === "all" ||
      getAvailabilityLevel(station.available_bikes) === availability;

    return matchesSearch && matchesAvailability;
  });

  displayStations(filteredStations);
}
```

Finally, attach event listeners so the map updates whenever the user changes the search or filter controls.

JavaScript

```javascript
let allStations = [];

fetch("/api/stations")
  .then(response => response.json())
  .then(data => {
    allStations = data;
    displayStations(allStations);
  });

document.getElementById("stationSearch").addEventListener("input", () => {
  filterStations(allStations);
});

document.getElementById("availabilityFilter").addEventListener("change", () => {
  filterStations(allStations);
});
```

Overall, this design makes the map easy to understand at a glance: green markers show stations with many bikes, orange markers show moderate availability, and red markers highlight stations with low availability. Popups provide more detailed information, while filtering helps users quickly find suitable stations.

Here's a clean template you can adapt for your bike station map.

Leaflet's basic flow is: create the map, add a tile layer, add markers, bind popups, then refresh the marker layer when data changes. Leaflet's current stable docs are for 1.9.4.

1. HTML

HTML

```
<link
  rel="stylesheet"
  href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"
/>

<div id="map"></div>

<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="map.js"></script>
```

2. CSS

CSS

```
#map {
  height: 600px;
  width: 100%;
  border-radius: 16px;
}
```

3. JavaScript

JavaScript

```
// Centre map on Dublin
const map = L.map("map").setView([53.3498, -6.2603], 13);

// Add OpenStreetMap tiles
L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution: "&copy; OpenStreetMap contributors"
}).addTo(map);

// Store markers in a layer group so they can be cleared/updated easily
const stationLayer = L.layerGroup().addTo(map);

// Example data
let stations = [
  {
    name: "Station A",
    address: "Dublin City Centre",
    lat: 53.3498,
    lng: -6.2603,
    available_bikes: 12,
    available_stands: 8
  },
  {
    name: "Station B",
    address: "Trinity College",
    lat: 53.3438,
    lng: -6.2546,
    available_bikes: 3,
    available_stands: 17
  }
];

function getMarkerColour(bikes) {
  if (bikes >= 15) return "green";
  if (bikes >= 5) return "orange";
  return "red";
}

function createMarkerIcon(colour) {
  return L.divIcon({
    className: "custom-marker",
    html: `<div style="
      background:${colour};
      width:18px;
      height:18px;
      border-radius:50%;
      border:2px solid white;
      box-shadow:0 2px 8px rgba(0,0,0,0.3);
    "></div>`,
    iconSize: [18, 18],
    iconAnchor: [9, 9]
  });
}

function renderStations(stationData) {
  stationLayer.clearLayers();

  stationData.forEach(station => {
    const colour = getMarkerColour(station.available_bikes);

    const marker = L.marker([station.lat, station.lng], {
      icon: createMarkerIcon(colour)
    });

    marker.bindPopup(`
      <strong>${station.name}</strong><br>
      ${station.address}<br><br>
      Bikes available: ${station.available_bikes}<br>
      Stands available: ${station.available_stands}
    `);

    marker.addTo(stationLayer);
  });
}

renderStations(stations);
```

4. Dynamic data update from an API

JavaScript

```
async function fetchStationData() {
  try {
    const response = await fetch("/api/stations");
    const data = await response.json();

    renderStations(data);
  } catch (error) {
    console.error("Could not update station data:", error);
  }
}

// Initial load
fetchStationData();

// Refresh every 5 minutes
setInterval(fetchStationData, 5 * 60 * 1000);
```

The important part is `stationLayer.clearLayers()`. It removes the old markers before adding updated ones, so you do not get duplicate markers every time the data refreshes. Leaflet layer groups are designed for managing multiple layers together.

Sources

How can I design a responsive layout for a web app where UI panels overlay an interactive map without breaking on smaller screens?

Use a **map wrapper** with absolutely positioned overlay panels, then switch panel sizes/positions at smaller breakpoints instead of letting them push the map around.

HTML

```html
<div class="map-page">
  <div class="map-shell">
    <div id="map"></div>

    <aside class="map-panel station-panel">
      Station details here
    </aside>

    <aside class="map-panel filters-panel">
      Filters here
    </aside>

    <button class="chatbot-button">💬</button>
  </div>
</div>
```

CSS

```css
.map-page {
  height: calc(100vh - 80px); /* adjust if you have a header */
  padding: 1rem;
}

.map-shell {
  position: relative;
  height: 100%;
  min-height: 600px;
  border-radius: 24px;
  overflow: hidden;
}

#map {
  height: 100%;
  width: 100%;
}

/* Shared panel styling */
.map-panel {
  position: absolute;
  z-index: 500;
  background: white;
  border-radius: 18px;
  padding: 1rem;
  box-shadow: 0 8px 28px rgba(0, 0, 0, 0.16);
  max-height: calc(100% - 2rem);
  overflow-y: auto;
}

/* Desktop layout */
.station-panel {
  top: 1rem;
  left: 1rem;
  width: 340px;
}

.filters-panel {
  top: 1rem;
  right: 1rem;
  width: 300px;
}

.chatbot-button {
  position: absolute;
  right: 1rem;
  bottom: 1rem;
  z-index: 600;
}

/* Laptop / smaller desktop */
@media (max-width: 1100px) {
  .station-panel {
    width: 300px;
  }

  .filters-panel {
    width: 260px;
  }
}

/* Tablet layout: keep overlays, but make them more compact */
@media (max-width: 850px) {
  .map-shell {
    min-height: 700px;
  }

  .station-panel {
    top: 0.75rem;
    left: 0.75rem;
    right: 0.75rem;
    width: auto;
    max-height: 220px;
  }

  .filters-panel {
    top: auto;
    left: 0.75rem;
    right: 0.75rem;
    bottom: 0.75rem;
    width: auto;
    max-height: 180px;
  }

  .chatbot-button {
    right: 1rem;
    bottom: 13rem; /* prevents overlap with bottom panel */
  }
}

/* Very small screens */
@media (max-width: 520px) {
  .map-page {
    padding: 0.5rem;
  }

  .map-panel {
    padding: 0.8rem;
    border-radius: 14px;
  }

  .station-panel {
    max-height: 200px;
  }

  .filters-panel {
    max-height: 160px;
  }
}
```

Main idea:

Keep the map as the full background layer. Put panels inside the same `.map-shell`, use `position: absolute`, and control overlap with `z-index`, `max-height`, and `overflow-y: auto`.

For smaller screens, avoid changing panels to normal document flow unless you actually want them above/below the map. Instead, make them full-width overlays at the top and bottom of the map. This keeps the app feeling like a proper interactive map interface.

Share ···

Can you give me a full JavaScript example of a Leaflet map that loads bike station data from a JSON file, adds markers, and displays popups with available bikes and stands?

Absolutely — here is a full `map.js` example you could use for a Dublin Bikes Leaflet map.

JavaScript

```javascript
// Create the Leaflet map and centre it on Dublin
const map = L.map("map").setView([53.3498, -6.2603], 13);

// Add OpenStreetMap tile layer
L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution: "&copy; OpenStreetMap contributors"
}).addTo(map);

// Store markers so they can be updated later
let stationMarkers = [];

// Function to choose marker colour based on available bikes
function getMarkerColour(availableBikes) {
  if (availableBikes >= 15) {
    return "green";
  } else if (availableBikes >= 5) {
    return "orange";
  } else {
    return "red";
  }
}

// Function to load station data from JSON
async function loadBikeStations() {
  try {
    const response = await fetch("bike_data.json");

    if (!response.ok) {
      throw new Error("Could not load bike station data");
    }

    const stations = await response.json();

    // Remove old markers before adding updated ones
    stationMarkers.forEach(marker => map.removeLayer(marker));
    stationMarkers = [];

    stations.forEach(station => {
      const lat = station.position.lat;
      const lng = station.position.lng;

      const availableBikes = station.available_bikes;
      const availableStands = station.available_bike_stands;

      const markerColour = getMarkerColour(availableBikes);

      const marker = L.circleMarker([lat, lng], {
        radius: 8,
        color: markerColour,
        fillColor: markerColour,
        fillOpacity: 0.85,
        weight: 2
      }).addTo(map);

      marker.bindPopup(`
        <div class="station-popup">
          <h3>${station.name}</h3>
          <p><strong>Address:</strong> ${station.address}</p>
          <p><strong>Available bikes:</strong> ${availableBikes}</p>
          <p><strong>Available stands:</strong> ${availableStands}</p>
          <p><strong>Status:</strong> ${station.status}</p>
        </div>
      `);

      stationMarkers.push(marker);
    });

  } catch (error) {
    console.error("Error loading bike station data:", error);
  }
}

// Load stations when the page opens
loadBikeStations();

// Refresh the data every 5 minutes
setInterval(loadBikeStations, 5 * 60 * 1000);
```

Your HTML needs a map container like this:

HTML

```html
<div id="map"></div>
```

And your CSS needs a height set:

CSS

```css
#map {
  height: 600px;
  width: 100%;
}
```

Example JSON structure:

JSON

```json
[
  {
    "name": "SMITHFIELD NORTH",
    "address": "Smithfield North",
    "position": {
      "lat": 53.349562,
      "lng": -6.278198
    },
    "available_bikes": 12,
    "available_bike_stands": 18,
    "status": "OPEN"
  }
]
```

Ask anything Instant

Share ···

How can I implement an autocomplete search bar in JavaScript that suggests bike station names as the user types?

You can do it by listening for `input` events, filtering your station list, and showing matching results in a dropdown.

HTML

```html
<div class="search-box">
  <input
    type="text"
    id="stationSearch"
    placeholder="Search for a bike station..."
    autocomplete="off"
  />
  <ul id="suggestions"></ul>
</div>
```

CSS

```css
.search-box {
  position: relative;
  width: 320px;
}

#stationSearch {
  width: 100%;
  padding: 0.75rem;
  border: 1px solid #ccc;
  border-radius: 8px;
}

#suggestions {
  position: absolute;
  top: 100%;
  left: 0;
  right: 0;
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  list-style: none;
  padding: 0;
  margin: 0.25rem 0 0;
  max-height: 220px;
  overflow-y: auto;
  z-index: 1000;
}

#suggestions li {
  padding: 0.7rem;
  cursor: pointer;
}

#suggestions li:hover {
  background: #edf8f0;
}
```

JavaScript

```javascript
const stations = [
  { name: "Smithfield North", address: "Smithfield" },
  { name: "Parnell Square North", address: "Parnell Square" },
  { name: "St. Stephen's Green East", address: "St. Stephen's Green" },
  { name: "Heuston Station", address: "Heuston" }
];

const searchInput = document.getElementById("stationSearch");
const suggestionsList = document.getElementById("suggestions");

searchInput.addEventListener("input", function () {
  const query = searchInput.value.toLowerCase().trim();
  suggestionsList.innerHTML = "";

  if (query === "") {
    return;
  }

  const matches = stations
    .filter(station =>
      station.name.toLowerCase().includes(query) ||
      station.address.toLowerCase().includes(query)
    )
    .slice(0, 6);

  matches.forEach(station => {
    const item = document.createElement("li");
    item.textContent = `${station.name} – ${station.address}`;

    item.addEventListener("click", function () {
      searchInput.value = station.name;
      suggestionsList.innerHTML = "";

      // Optional: call another function here
      // e.g. selectStation(station);
      console.log("Selected station:", station);
    });

    suggestionsList.appendChild(item);
  });
});
```

For your Dublin Bikes map, you could replace the sample `stations` array with data loaded from your JSON/API, then call something like:

JavaScript

```javascript
selectStation(station);
map.setView([station.latitude, station.longitude], 16);
```

when a user clicks a suggestion.

Ask anything Instant

Share ···

Can you write a JavaScript function to colour Leaflet markers based on bike availability levels (high, medium, low) and explain the thresholds?

Absolutely — this is a really nice UX touch for your map 👀 (and you're already using something very close to this based on your project notes).

✅ 1. Define availability thresholds

A simple and intuitive approach (what most bike apps use):

- **High availability (green)** → plenty of bikes
  → `>= 15 bikes`
- **Medium availability (orange)** → usable but limited
  → `5–14 bikes`
- **Low availability (red)** → almost empty
  → `< 5 bikes`

These thresholds work well because:

- `< 5` → user risks station being empty
- `5–14` → moderate reliability
- `>= 15` → safe choice

You can tweak these later based on station capacity if you want something more dynamic.

✅ 2. JavaScript function for marker colour

Here's a clean, reusable function:

JavaScript

```
function getMarkerColor(availableBikes) {
  if (availableBikes >= 15) {
    return "#1a9632"; // green (high)
  } else if (availableBikes >= 5) {
    return "#f0ad4e"; // orange (medium)
  } else {
    return "#d9534f"; // red (low)
  }
}
```

✅ 3. Apply it to Leaflet markers

If you're using circle markers (recommended for styling):

JavaScript

```
function createBikeMarker(station) {
  const color = getMarkerColor(station.available_bikes);

  return L.circleMarker([station.lat, station.lng], {
    radius: 8,
    fillColor: color,
    color: "#333",       // border colour
    weight: 1,
    opacity: 1,
    fillOpacity: 0.85
  }).bindPopup(`
    <strong>${station.name}</strong><br>
    Bikes: ${station.available_bikes}<br>
    Stands: ${station.available_bike_stands}
  `);
}
```

✅ 4. Optional: Add a status label (nice UX boost)

You can reuse the same logic for text:

JavaScript

```
function getAvailabilityStatus(bikes) {
  if (bikes >= 15) return "High";
  if (bikes >= 5) return "Moderate";
  return "Low";
}
```

Then in popup:

JavaScript

```
Status: ${getAvailabilityStatus(station.available_bikes)}
```

🔥 5. (Optional upgrade) Make it relative to station capacity

Right now it's fixed thresholds — but a more *data-driven* version:

JavaScript

```
function getMarkerColorRelative(bikes, capacity) {
  const ratio = bikes / capacity;

  if (ratio >= 0.6) return "#1a9632";   // high
  if (ratio >= 0.3) return "#f0ad4e";   // medium
  return "#d9534f";                     // low
}
```

This is actually *better* for real-world accuracy because:

- 10 bikes at a 15-capacity station = high
- 10 bikes at a 40-capacity station = low

💡 What I'd recommend for your project

Given your current implementation + deadlines:

- ✅ Stick with **fixed thresholds** for now (simple, clear, consistent)
- ⭐ Mention **relative thresholds as a future improvement** in your report (this scores well 👀)

If you want next step, I can help you:

- add a **legend UI (green/orange/red meaning)**
- or animate markers updating every refresh (looks *very* impressive in demos)